

Frontend Notes

Nepal AI Travel Planner: how the website works, explained from zero

These notes walk through every file that makes up the website. Read them in order. Each section explains one idea, shows real code from **your** project, and suggests things to try. No earlier frontend knowledge is needed.

> **What was built**
> A one-page website with a trip builder: pick a destination, drag sliders for days and budget, choose travellers, road or flight, and interests. The page writes the request for you, sends it to the LangGraph agents in `backend.py`, and shows the plan with a clickable table of contents. The design uses a calm "Himalayan Lake" palette, your Everest and prayer-flag photos, and subtle motion.

Section	Topic
1	The big picture: frontend vs backend
2	The files and what each one does
3	HTML: the structure of the page
4	CSS: how the page looks (design tokens, layout, images)
5	JavaScript: the trip builder and talking to the server
6	Effects and motion: transitions, scroll reveal, parallax
7	app.py: the bridge between the page and your agents

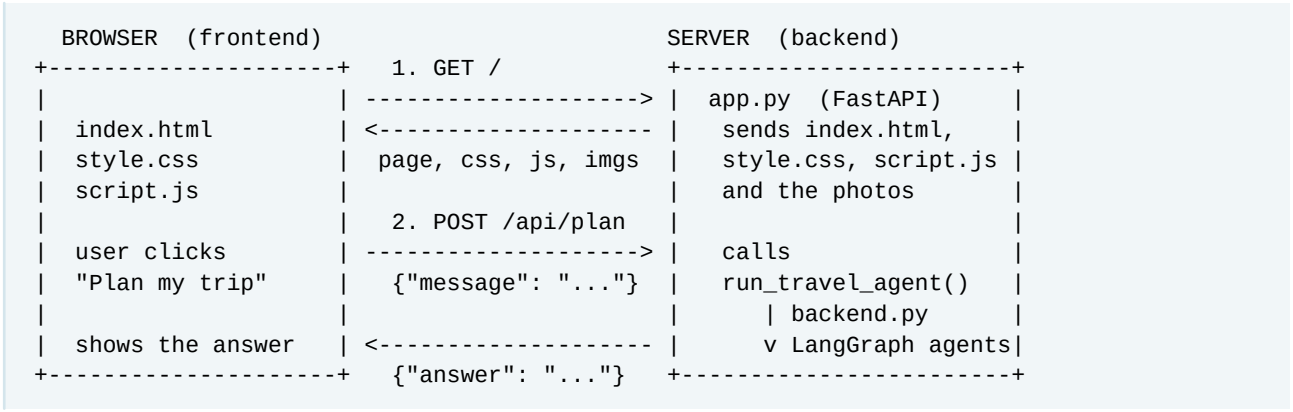
8	The journey of one click, step by step
9	Running it, and practice exercises
10	Glossary

1. The big picture

Every website has two halves that live in different places:

	Frontend	Backend
Where it runs	In the visitor's **browser** (Chrome, Edge...)	On a **server** (your computer for now)
What it does	Shows things, reacts to clicks	Does the heavy work: AI calls, database, searches
Languages	HTML, CSS, JavaScript	Python (FastAPI + LangGraph in your project)
Your files	templates/index.html, static/style.css, static/script.js, static/images/	app.py, backend.py, tools/

They talk over **HTTP**, the same thing your browser uses to open any website. The browser sends a **request** ("give me the page" or "here is a trip, plan it") and the server sends back a **response**.



> **Think of a restaurant**
> The **frontend** is the dining room and the menu: what customers see and touch. The **backend** is the kitchen. **HTTP requests** are the waiter carrying orders in and food out. `app.py` is the waiter.

2. The files

File	Job	Analogy
`templates/index.html`	**Structure**: what is on the page (nav, hero, builder, results)	The skeleton
`static/style.css`	**Looks**: colours, fonts, spacing, layout, animations	Clothes and paint
`static/script.js`	**Behaviour**: the trip builder, talking to the server, scroll effects	The muscles
`static/images/`	Your photos (everest.jpg, nepal-2.jpg) and the small tab icon (favicon.svg)	The pictures on the wall

`app.py`	**Server**: sends the page, receives trip requests, calls your agents	The waiter

Why these folder names? FastAPI (with its helper Jinja2) looks for HTML pages in `templates`. Files sent to the browser unchanged (CSS, JavaScript, images) go in `static`. Your photos were copied there with simple names, because spaces and emoji in file names cause trouble in URLs.

3. HTML: the structure

HTML is a list of **elements**. Each one has an opening tag and a closing tag, with content between:

```
<h1>Plan your journey through Nepal</h1>
 ^^^ opening tag    content          ^^^^^ closing tag (note the /)
```

Tags can have **attributes**, extra settings inside the opening tag:

```
<button type="button" class="chip" data-place="Chitwan">Chitwan</button>
        ^^^^^^^^^^^^^ ^^^^^^^^^^^^ ^^^^^^^^^^^^^^^^^^^^
        not a submit   style group  our own data for JavaScript
```

> **id, class and data-**
> **id** is a *unique* name for one element, so JavaScript can find it: `document.getElementById("days")`.
> **class** is a *group* name shared by many elements, so CSS can style them the same way.
> **data-anything** stores your own information on an element. JavaScript reads it as `chip.dataset.place` (here "Chitwan"). That's how one piece of code can handle all seven destination buttons.

The tags used in your page

Tag	Meaning
`<head>`	Invisible setup: title, fonts, CSS link, tab icon
`<nav>`	The navigation bar at the top
`<header> <section> <aside> <footer>`	Labelled areas: top area, content chunk, side column, bottom
`<h1> <h2> <h3> <p>`	Headings from biggest to smallest, and paragraphs
`<figure> <img> <figcaption>`	A photo with its caption. **alt** describes the photo for blind users and if it fails to load
`<form>`	Groups the inputs; pressing the submit button fires a "submit" event
`<input type="text">`	One-line text box (Starting from / Going to)
`<input type="range">`	A slider with min, max, step and value (days, budget)
`<output>`	Shows a calculated value ("5 days", "Rs 40,000")
`<textarea>`	Multi-line text box (Your request)
`<dl> <dt> <dd>`	A list of label + value pairs (the trip summary)
`<ol> <li>`	A numbered list (the four steps, the loading stages)
`<svg> <path>`	A drawing made of lines: the mountain ridge in the loading screen
`<script>`	Load or run JavaScript

The page, top to bottom

```
<nav class="nav">            brand on the left, two links on the right
<header class="hero">        headline + buttons | Everest photo (full, uncropped)
<section id="how">           the four steps (matches the 4 agents)
<section id="planner">
   <form class="builder">    inputs, chips, sliders, stepper, toggle, textarea
   <aside class="summary">   prayer-flag photo + live "trip at a glance"
<section id="loading">       ridge drawing + 4 stages     (hidden at first)
<section id="error">         error message                (hidden at first)
<section id="result">        table of contents | the answer (hidden at first)
<footer>                     shubha yatra (have a good journey)
<script src="/static/script.js">   loaded LAST, after everything exists
```

The trick behind the whole page: "hidden"

The loading, error and result sections are **always in the HTML**, but start with class `hidden`. CSS says `.hidden { display: none }`. JavaScript **removes** that class to show a section and **adds** it back to hide it. No new pages are loaded: we just switch sections on and off.

Links that jump within the page

`<a href="#planner">` jumps to the element with `id="planner"`. `html { scroll-behavior: smooth }` in the CSS makes it glide instead of jump.

4. CSS: the looks

A CSS **rule** = a **selector** (which elements) + **properties** (what to change):

```
.builder {                          <- selector: elements with class="builder"
  background: var(--white);         <- property: value;
  border: var(--line);              <- thin ice-blue border
  border-radius: var(--radius);     <- slightly rounded corners (6px)
  padding: var(--s5);               <- space INSIDE the box (48px)
}
```

Selectors you'll see

Selector	Picks
`body`	the <body> tag (tag name, no symbol)
`.chip`	everything with class="chip" (dot = class)
`#planner`	the one element with id="planner" (hash = id)
`.answer h2`	<h2> tags *inside* .answer (space = "inside")
`.chip.is-selected`	a chip that *also* has class is-selected (no space = both)
`.chip:hover`	a chip *while the mouse is over it*
`hr + h2`	an h2 that comes *right after* an hr
`:focus-visible`	whatever you reached with the Tab key (the keyboard focus ring)

Design tokens: the whole look in one place

At the top of style.css, `:root` defines **variables** (names starting with --). Everything else uses them with `var()`:

```
:root {
  --deep:    #193E55;   /* Deep Himalayan Blue: headings, main buttons */
  --lake:    #39718B;   /* Lake Blue: links, selected things           */
  --glacier: #8DB7C9;   /* Glacier Blue: decoration only               */
  --ice:     #D4E5EC;   /* Ice Blue: thin borders                      */
  --mist:    #F1F6F8;   /* Snow Mist: page background                  */

  --serif: "Cormorant Garamond", Georgia, serif;   /* headings */
  --sans:  "DM Sans", system-ui, sans-serif;       /* everything else */

  --s1: 8px; --s2: 16px; --s3: 24px; --s4: 32px;   /* spacing, all x8 */
  --s5: 48px; --s6: 64px; --s7: 96px;
  --radius: 6px;
}
```

- **Hex colours**: # then two digits each for red, green, blue (00 to ff). #193E55 = a little red, more green, most blue.
- **Font lists**: if Cormorant Garamond fails to load, the browser tries Georgia, then any serif font.
- **8px spacing**: every gap is a multiple of 8, so spacing feels consistent without you thinking about it.

- **Contrast**: Glacier Blue is too light for small text, so it is only used for decoration. Text uses --deep, --text or --muted, which are dark enough to read easily (the WCAG AA standard).

The box model

```
 +---------------- margin (space outside, pushes others away) ---+
 |  +------------- border (the visible edge line) ------------+  |
 |  |  +---------- padding (space inside, around content) -+  |  |
 |  |  |              CONTENT  (text, etc.)              |  |  |
 |  |  +-------------------------------------------------+  |  |
 |  +-------------------------------------------------------+  |
 +-------------------------------------------------------------+
```

Layout: Grid and Flexbox

CSS Grid divides an area into columns. The hero, planner, steps and results all use it:

```
.hero {
  display: grid;
  grid-template-columns: 1.15fr 0.85fr;   /* two columns, left a bit wider */
  gap: 64px;                              /* space between them */
}
.steps { grid-template-columns: repeat(4, 1fr); }   /* four equal columns */
```

`fr` means "a fraction of the free space". **Flexbox** (`display: flex`) is for one row of things, like the chips or the buttons. `flex-wrap: wrap` lets them drop to a new line when space runs out.

Responsive design: media queries

```
@media (max-width: 960px) {                       /* tablets and phones */
  .hero         { grid-template-columns: 1fr; }  /* one column: photo below text */
  .planner-grid { grid-template-columns: 1fr; }
}
@media (max-width: 600px) {                       /* phones */
  .hero h1 { font-size: 44px; }
}
```

Rules inside `@media` only apply when the condition is true. `clamp(44px, 6.4vw, 80px)` is another trick: grow with screen width (`vw` = 1% of it), never smaller than 44px or bigger than 80px.

Images

```
/* Everest: shown whole, in its original tall shape */
.hero-figure { max-width: 400px; }                 /* cap the width...       */
.hero-photo img { width: 100%; height: auto; }     /* ...height follows it   */

/* Prayer flags: cropped to fill a fixed shape */
.summary-photo { aspect-ratio: 16 / 10; overflow: hidden; }
.summary-photo img { width: 100%; height: 100%; object-fit: cover; }
```

- **height: auto** keeps a photo's own proportions. The Everest photo is 736 x 1307 pixels, so at 400px wide it is about 710px tall. Nothing is cut off.
- **object-fit: cover** does the opposite: it fills a box of a fixed shape completely, without stretching, by cropping the edges. The prayer-flag photo uses this.

- `aspect-ratio: 16 / 10` gives a box a fixed shape (16 wide for every 10 tall) whatever its width.
- Putting `width` and `height` on the `<img>` tag lets the browser reserve the right space before the photo downloads, so the page doesn't jump.

Patterns without images

The brand mountain is a dark square cut to shape with `clip-path: polygon(...)`. The arrow in "Kathmandu → Pokhara" is a 1px line plus a rotated corner border. No image files are needed for either.

5. JavaScript: behaviour

JavaScript makes the page react. Almost everything in script.js follows one pattern:

> **The 3-step pattern**
> **1. Find** an element → **2. Listen** for an event on it (click, input, submit) → **3. Do something** (change text, toggle a class, send a request).

Step 1: find elements

```
const $ = (id) => document.getElementById(id);   // a short helper
const daysInput = $("days");
```

`document` is the whole page. `const` creates a variable that won't be reassigned; `let` is for ones that will. `(id) => ...` is an **arrow function**, a small function like Python's `lambda`.

Step 2 and 3: listen and react

```
document.querySelectorAll("#destinations .chip").forEach((chip) => {
  chip.addEventListener("click", () => {
    toInput.value = chip.dataset.place;   // "Chitwan" from data-place
    update();
  });
});
```

`querySelectorAll` finds every element matching a CSS selector. `forEach` loops over them like a Python for-loop. The inner function runs later, each time that chip is clicked.

The builder's big idea: one state, one update()

All the trip choices live in one object, and every control calls the same function after changing something:

```
const trip = { travellers: 2, mode: "bus", interests: [] };

function update() {
  const days = Number(daysInput.value);          // slider value is text; make it a number
  const budget = Number(budgetInput.value);
  setText($("days-out"), `${days} days`);       // live label beside the slider
  setText($("sum-ppd"), formatRs(budget / (days * trip.travellers)));
  if (!requestEdited) {                          // don't overwrite the user's own words
    messageBox.value = `${days} day trip from ${from} to ${to} ...`;
  }
}
daysInput.addEventListener("input", update);     // "input" fires while dragging
```

- **Template strings**: text in backticks can contain `${...}`, just like Python f-strings.
- **Why one update()?** Every display is recalculated from the same data, so the slider, the summary and the request text can never disagree.
- **Number formatting**: `toLocaleString("en-IN")` writes 150000 as 1,50,000, the lakh style used in Nepal.

- **Flights**: choosing Flight adds the word "fly" to the request. Your backend.py looks for that word and only then calls the flight API.
- **requestEdited**: once you type in the request box, update() stops overwriting it. "Rewrite from options" turns it back on.

Talking to the server with fetch

```
form.addEventListener("submit", async (event) => {
  event.preventDefault();          // stop the browser reloading the page
  setLoading(true);
  try {
    const response = await fetch("/api/plan", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ message: message, thread_id: threadId }),
    });
    const data = await response.json();
    if (data.error) showError(data.error);
    else            showAnswer(data.answer, title);
  } catch (err) {
    showError("Could not reach the server. Is it running?");
  } finally {
    setLoading(false);
  }
});
```

- **preventDefault()**: a form normally reloads the page when submitted. We cancel that and handle it ourselves.
- **fetch**: sends an HTTP request. `POST` means "I'm sending data". It reaches the `/api/plan` route in app.py.
- **JSON**: a text format for data, like a Python dict. `JSON.stringify` turns an object into text; `response.json()` turns the reply back.
- **async / await**: the AI takes about 30 seconds. `await` waits *without freezing the page*, so the loading animation keeps moving.
- **try / catch / finally**: the same as Python's try / except / finally.

Showing the answer: Markdown, emoji and safety

```
const clean = cleanAnswer(markdownText);
answerBox.innerHTML = DOMPurify.sanitize(marked.parse(clean));
```

- The AI replies in **Markdown** (`## Heading`, `**bold**`, | tables |). The **marked** library turns it into HTML.
- **cleanAnswer** uses **regular expressions** (text patterns, like Python's `re` module). `\p{Emoji_Presentation}` matches characters that show as colour emoji; plain symbols like → are kept. A second pattern turns long dashes into a normal hyphen.
- Codes like `\uFE0F` are a way to write invisible or special characters by number, so you can see them in the code.

- `innerHTML` puts HTML inside an element. If the text contained a `<script>` tag, it could run code in your visitor's browser, so **DOMPurify** removes anything dangerous first. For plain text, `textContent` is always safe.

The table of contents

After the answer is shown, the code finds every `h2`, gives each an id (`section-1`, `section-2`...) and builds a link list with `document.createElement`. Clicking a link jumps to that id. Which link is highlighted is decided by an IntersectionObserver (next section).

Remembering the conversation: localStorage

The backend stores memory in PostgreSQL by `thread_id`. The page keeps that id in **localStorage**, a small store inside the browser that survives refreshes. "Start over" deletes it. Each call is wrapped in `try { } catch (e) { }` because some browsers block it in private mode.

6. Effects and motion

Motion here is small and has a purpose: it shows that something responded, or guides your eye. Five techniques make up all of it.

Transitions: smooth changes

```
.chip { transition: background-color 180ms, transform 180ms; }
.chip:hover { transform: translateY(-1px); }   /* lift 1px on hover */
.chip.is-selected { background: var(--deep); color: white; }
```

Without `transition`, a change happens instantly. With it, the browser animates between the old and new value over 180 milliseconds. `transform` moves (`translateY`), grows (`scale`) or turns (`rotate`) an element without moving anything around it.

Keyframe animations: things that move by themselves

```
.ridge path {
  stroke-dasharray: 340;  stroke-dashoffset: 340;   /* line fully "hidden" */
  animation: draw 2.4s infinite;
}
@keyframes draw {
  60%  { stroke-dashoffset: 0; }      /* fully drawn */
  100% { stroke-dashoffset: -340; }   /* slides away */
}
```

The mountain ridge on the loading screen "draws itself" by animating where a dashed line's single long dash begins. The pulsing circle on the active loading stage and the small bump when a number changes are also keyframes.

Scroll reveal: IntersectionObserver

```
const revealObserver = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {                    // it came on screen
      entry.target.classList.add("is-visible");    // CSS fades it in
      revealObserver.unobserve(entry.target);      // only once
    }
  });
}, { threshold: 0.15 });                           // when 15% is visible
```

An IntersectionObserver tells you when an element enters the screen. CSS does the rest: `.reveal` starts transparent and 20px low, and `.reveal.is-visible` brings it to full opacity in place. The four steps each get a slightly longer `transitionDelay`, so they appear one after another. The same tool highlights the current section in the table of contents.

> **Why the html.js class?**
> A one-line script in <head> adds class `js` to the page. The hiding CSS is written as `.js .reveal`, so if JavaScript ever fails to load, nothing stays invisible.

Parallax: depth while scrolling

```
window.addEventListener("scroll", () => {
  requestAnimationFrame(() => {
    heroPhoto.style.transform = `translateY(${-window.scrollY * 0.08}px)`;
  });
});
```

The whole Everest photo drifts up at 8% of the scroll speed, slower than the page around it, which feels like depth. `requestAnimationFrame` waits for the browser's next repaint, which keeps it smooth instead of running hundreds of times a second.

Respecting people who turn motion off

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation: none !important; transition: none !important; }
}
```

Some people get dizzy from animation and switch it off in Windows, macOS or their phone settings. This rule (and a matching check in script.js) turns every effect off for them. Always include it.

7. app.py: the bridge

```
app = FastAPI(title="Nepal AI Travel Planner")
app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

class PlanRequest(BaseModel):          # the shape of data we expect
    message: str
    thread_id: str | None = None

@app.get("/")                          # route 1: open the website
def home(request: Request):
    return templates.TemplateResponse(request, "index.html")

@app.post("/api/plan")                 # route 2: plan a trip
def plan_trip(body: PlanRequest):
    result = run_travel_agent(message, body.thread_id)
    return {"thread_id": result["thread_id"], "answer": result["answer"]}
```

- **@app.get("/")**: a *decorator* meaning "when a browser asks for / with GET, run this function". It returns your HTML page.
- **app.mount("/static", ...)**: any URL starting with /static/ is served straight from the static folder. That's how `/static/images/everest.jpg` works.
- **PlanRequest(BaseModel)**: Pydantic checks the incoming JSON has a `message` string; FastAPI rejects it otherwise.
- **Returning a dict**: FastAPI converts it to JSON. That's the data object in script.js.
- **def, not async def**: your agent code is slow and blocking. A plain def makes FastAPI run it in a separate thread, so the server stays responsive.

> **Bonus**
> With the server running, open `http://127.0.0.1:8000/docs` to see and test `/api/plan` directly in the browser.

8. The journey of one click

#	Where	What happens
1	Browser	You open http://127.0.0.1:8000. The browser sends **GET /**.
2	app.py	`home()` returns index.html.
3	Browser	Reads the HTML and requests the CSS, JS, fonts and photos it mentions. The page appears.
4	script.js	Runs once: attaches listeners, calls update() to fill in the request, starts watching for scrolling.
5	You	Click Chitwan, drag the sliders, toggle Wildlife. Each change calls update(), and the summary and request text change instantly.
6	You	Press **Plan my trip**. The form fires a *submit* event.
7	script.js	Cancels the reload, shows the loading section, sends **POST /api/plan** with {message, thread_id}.
8	app.py	`plan_trip()` calls `run_travel_agent()`.
9	backend.py	transport_agent → hotel_agent → itinerary_agent → final_agent (Tavily, flights, Groq). About 30 s.
10	app.py	Returns JSON {thread_id, answer}.
11	script.js	Strips emoji, converts Markdown to safe HTML, builds the table of contents, shows the result and scrolls to it.

9. Running it

```
# 1. activate your virtual environment (Windows PowerShell)
.\travel\Scripts\Activate.ps1

# 2. start the server (--reload restarts it when you save a Python file)
uvicorn app:app --reload

# 3. open in your browser
http://127.0.0.1:8000
```

- Stop the server with **Ctrl + C**.
- After changing HTML, CSS or JS, just **refresh**. If a change doesn't show, press **Ctrl + Shift + R** (a hard refresh that skips the cache).
- Press **F12** for **Developer Tools**. *Console* shows JavaScript errors; *Network* shows every request (click /api/plan to see what was sent and received); *Elements* lets you edit HTML/CSS live. The phone icon in the top-left previews phone sizes.
- Python errors (like a wrong API key) appear in the **terminal** running uvicorn, not in the browser.

Practice exercises (easiest first)

- **Text**: change the h1 in index.html to your own words. Save, refresh.
- **Colour**: in style.css change `--deep` to #2E4A3A (a forest green). Every heading and main button changes. Change it back after.
- **Corners**: change `--radius: 6px` to 0px, then to 14px. Notice how the mood changes.
- **New destination**: copy a `<button ... data-place="Mustang">` line, change both the data-place and the text to "Ilam". It works with no JavaScript changes. Why? (Hint: querySelectorAll.)
- **Longer trips**: change the days slider's `max="21"` to 30.
- **console.log**: add `console.log(trip);` as the first line inside update(). Open F12 → Console and click some chips. This is JavaScript's print().
- **Motion test**: in Windows Settings → Accessibility → Visual effects, turn off Animation effects, then reload. All motion should stop.
- **Challenge**: add a "Season" segmented toggle (Spring / Monsoon / Autumn / Winter) that adds "Travelling in autumn." to the request. Copy how trip.mode works.

10. Glossary

Word	Meaning
HTML / CSS / JavaScript	Structure / looks / behaviour of a web page
Element, tag, attribute	One piece of HTML; the <name> that makes it; a setting on it (id, class, src, alt)
DOM	The page as objects JavaScript can read and change (`document`)

Event	Something that happens: click, input, submit, scroll
State	The data that describes what's currently chosen (the trip object)
Design token	A named value (colour, size, spacing) reused everywhere: --deep, --s3
Grid / Flexbox	CSS layout systems: columns / one row of items
Media query	CSS that only applies at certain screen sizes or settings
Transition / keyframes	Smooth change between two looks / an animation with steps
Transform	Move, scale or rotate an element without affecting its neighbours
IntersectionObserver	Tells your code when an element enters or leaves the screen
Parallax	Layers moving at different speeds while scrolling, creating depth
HTTP, GET, POST	How browser and server talk; "give me" vs "here is data"
Route / endpoint / API	A URL the server answers; ones meant for programs return JSON
JSON	Text format for data, like a Python dict
Markdown	Simple text formatting (# heading, **bold**) the AI writes in
Regular expression	A pattern for finding text (used to remove emoji)
CDN	A public server hosting libraries and fonts we load by URL
localStorage	Small browser storage that survives refreshes
Accessibility (a11y)	Making the site usable for everyone: contrast, keyboard focus, alt text, reduced motion